

Yes — and you're right to stop here, because there are **three different things getting mixed together**: `handlers`, `auth`, and the Google credentials. Let's separate them. 😊

1. Is `handlers` imported from `@/handlers`?

No. In the setup we've been discussing, you have:

```
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({
  providers: [
    Google(...)
  ]
});
```

So `handlers` is exported from your `auth.ts` file.

Therefore:

```
// app/api/auth/[...nextauth]/route.ts

import { handlers } from "@auth";

export const { GET, POST } = handlers;
```

Notice:

`@/auth`



not:

`@/handlers`



Why?

Because `handlers` isn't a file you created.

It's one of the things **returned by** `NextAuth()`.

Think:

```
const result = NextAuth({...});

result = {
  handlers: ...,
  auth: ...,
  signIn: ...,
  signOut: ...
}
```

Then you do:

```
export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```

So all four are exported from **your** `auth.ts` .

2. So what exactly is `handlers` ?

This is worth making very concrete.

You write:

```
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```

Auth.js gives you a collection of functionality.

One piece is:

</> TypeScript



which contains the HTTP handlers Auth.js wants Next.js to use.

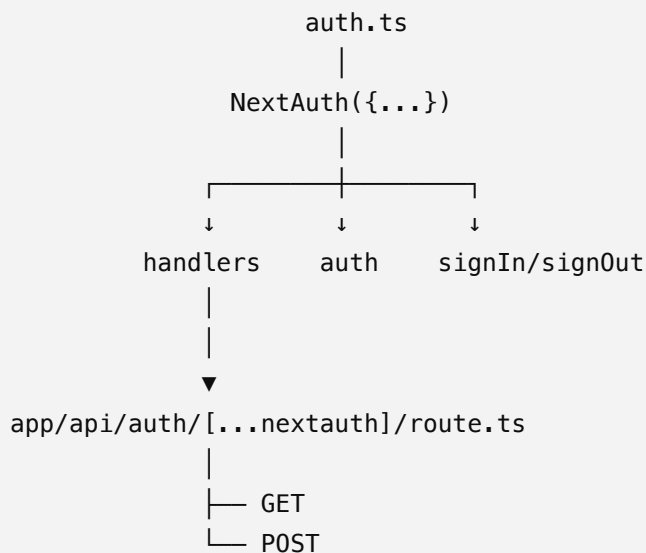
Then your Route Handler does:

```
import { handlers } from "@auth";  
  
export const { GET, POST } = handlers;
```

So you're essentially saying:

"Next.js, for this route, use the GET and POST functions that Auth.js gave me."

Therefore:



3. Now: What's the Client ID?

This is from **Google**, not Auth.js.

When you want Google Login, you have to tell Google:

"Hey Google, I have an application and I want users to be able to log in with their Google accounts."

You create/register an application in Google's developer console.

Google gives your application a:

Client ID



For example, conceptually:

1234567890-abc123xyz.apps.googleusercontent.com



The Client ID is basically:

"Which application is asking Google for authentication?"

It is **not** a password.

It's okay for the Client ID to be sent to the browser.

4. What is the Client Secret?

Google also gives your application:

Client Secret



Conceptually:

GOCSPX-xxxxxxxxxxxxxxxxxxxx



This is a **secret credential** that your server uses to prove to Google that it is the **registered application**.

So:

Client ID



Identifies your application.

Client Secret

Proves your server is allowed to act as that application.

Very important:

The secret should **NOT** be sent to browser JavaScript.

You'd have:

```
AUTH_GOOGLE_ID=1234567890-abc.apps.googleusercontent.com  
AUTH_GOOGLE_SECRET=G0CSPX-xxxxxxx
```

and Auth.js reads these on the **server**.

5. Why do we need both?

Imagine Google has registered:

Application: Gopi's Next.js App



Client ID:
123456.apps.googleusercontent.com

Client Secret:
SUPER_SECRET_123

Your application starts OAuth:

Browser
↓
Google



Google can see:

```
client_id = 123456.apps.googleusercontent.com
```



Google says:

"Okay, I know which application this is."

Later, your **server** exchanges the authorization code:

Your server
↓
Google



and sends credentials including:

```
client_id  
client_secret  
code  
redirect_uri
```



Google can verify:

"This server knows the secret belonging to this registered client."

That's one of the reasons the token exchange happens **server-side** in a confidential-server flow.

6. Now what's Google's Token Endpoint?

This is another URL operated by Google.

Earlier we had:

```
/api/auth/callback/google?code=ABC123
```



That's **YOUR application's callback endpoint**.

Don't confuse that with Google's token endpoint.

There are two different endpoints:

Your endpoint

```
https://yourapp.com/api/auth/callback/google
```



Purpose:

Google redirects the browser back to your application.

Google's token endpoint

Google has an endpoint whose job is:

"Give me this authorization code, and I'll exchange it for tokens."

Conceptually:

Your Next.js server

```
|  
| POST  
| code=ABC123  
| client_id=...  
| client_secret=...
```



Google token endpoint

```
|
```



Google returns tokens

For Google's OAuth implementation, the token endpoint is:

```
https://oauth2.googleapis.com/token
```



So don't think of it as another page the user visits.

It's an **OAuth server endpoint** used by your server/Auth.js to exchange the **authorization code** for **tokens**.

7. Let's see the entire thing with actual URLs

Suppose:

Your app:
`https://myapp.com`



Step 1 — Login starts

Your browser ultimately gets sent to Google's authorization endpoint:

```
https://accounts.google.com/o/oauth2/v2/auth
```



with parameters such as:

```
client_id=YOUR_CLIENT_ID  
redirect_uri=https://myapp.com/api/auth/callback/google  
response_type=code  
scope=openid profile email  
state=...
```



Step 2 — Google authenticates user

User logs into Google.

Google generates:

```
authorization code = ABC123
```



Step 3 — Google sends browser back

Browser goes:

```
https://myapp.com/api/auth/callback/google?code=ABC123
```



That's **your Next.js endpoint**.

Step 4 — Auth.js receives the code

Your:

```
app/api/auth/[...nextauth]/route.ts
```



catches it.

```
Next.js
```

```
↓
```

```
Auth.js
```



Step 5 — Auth.js talks directly to Google

The browser is no longer the important participant here.

Your server/Auth.js sends something conceptually like:

```
POST https://oauth2.googleapis.com/token

code=ABC123
client_id=YOUR_CLIENT_ID
client_secret=YOUR_CLIENT_SECRET
redirect_uri=https://myapp.com/api/auth/callback/google
grant_type=authorization_code
```

Google validates all of that.

Step 6 — Google returns tokens

Conceptually:

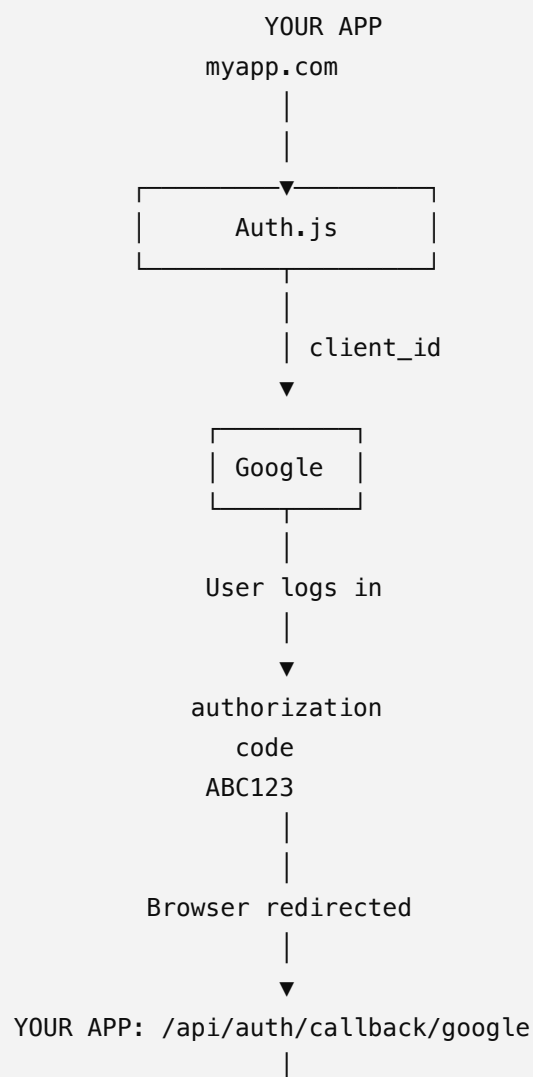
```
{  
  "access_token": "...",  
  "id_token": "...",  
  "expires_in": 3600  
}
```

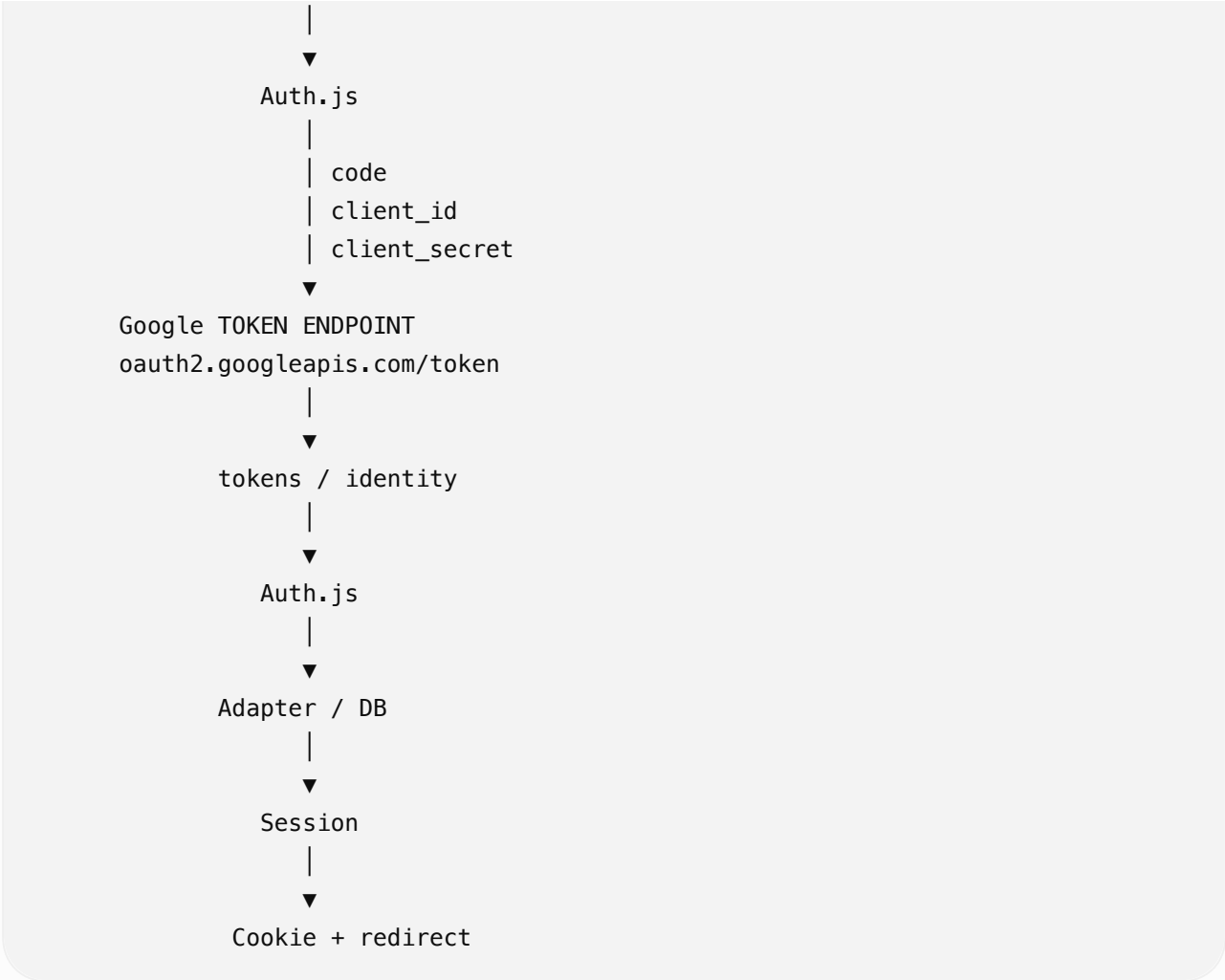
Auth.js can then use the OIDC identity information to determine:

```
Google  
  ↓  
providerAccountId = 123456789  
  ↓  
Your application's User  
  ↓  
Create session
```



8. One diagram to lock it in 🧠





The 4 URLs/identifiers you should distinguish

Thing	What it means
Client ID	Identifies your application to Google
Client Secret	Secret credential proving your server is the registered application
Your callback URL	<code>/api/auth/callback/google</code> — where Google sends the browser back
Google token endpoint	<code>https://oauth2.googleapis.com/token</code> — where Auth.js exchanges the code for tokens

And finally:

```
auth.ts
  ↓
NextAuth({...})
  ↓
{ handlers, auth, signIn, signOut }
```

You create `auth.ts` ; `Auth.js` creates/provides those functions. That's the clean mental model. 🔥



okay

Perfect 😊 We've got the OAuth pieces separated now:

```
auth.ts
  ↓
NextAuth()
  ↓
handlers / auth / signIn / signOut

Google
  ↓
Client ID = identifies your app
Client Secret = proves your server's identity

Google login
  ↓
authorization code
  ↓
/api/auth/callback/google
  ↓
Auth.js → Google token endpoint
  ↓
identity
  ↓
User/Account
  ↓
Session
  ↓
Cookie
  ↓
logged-in app
```



That's a solid foundation for the next `Auth.js` piece. 🧩



okay next

Awesome 😄 Let's move to the next important Auth.js concept: how the session is actually used to protect pages and APIs.

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

```
Google
  ↓
OAuth/OIDC
  ↓
Auth.js callback
  ↓
User/Account
  ↓
Session
  ↓
Cookie
```



Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/
├── login/
│   └── page.tsx
└── dashboard/
    └── page.tsx
```



Our dashboard should only be accessible to logged-in users.

We can write:

```
<> TypeScript

// app/dashboard/page.tsx

import { auth } from "@auth";
```



```
import { redirect } from "next/navigation";

export default async function DashboardPage() {
  const session = await auth();

  if (!session) {
    redirect("/login");
  }

  return (
    <div>
      <h1>Welcome {session.user?.name}</h1>
      <p>{session.user?.email}</p>
    </div>
  );
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http

GET /dashboard
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript

const session = await auth();
```



runs.

3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript
```




```
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```

So `auth()` is an Auth.js server-side function.

Conceptually:

```
graph TD
    Browser -- "Cookie: session-token=ABC123" --> Nextjs[Next.js]
    Nextjs --> Dashboard[Dashboard Server Component]
    Dashboard -- "auth()" --> Authjs[Auth.js]
    Authjs --> Session[Session verification / lookup]
    Session --> User[User session]
```

If the session is valid:

```
</> TypeScript

session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

```
</> TypeScript
```

```
const session = await auth();
```



returns something equivalent to:

```
</> TypeScript
```

```
null
```



So:

```
</> TypeScript
```

```
if (!session) {  
  redirect("/login");  
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http
```

```
307 Temporary Redirect  
Location: /login
```



The browser then requests:

```
</> http
```

```
GET /login
```



So:

```
User → /dashboard
```

```
↓
```

```
auth()
```

```
↓
```

```
no session
```

```
↓
```

```
redirect("/login")
```



↓
/login

5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript  
  
if (!session)
```



That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

```
</> TypeScript  
  
const session = await auth();  
  
if (!session) {  
  redirect("/login");  
}  
  
if (session.user.role !== "ADMIN") {  
  redirect("/forbidden");  
}
```



Now we're asking two different questions:

Authentication:
"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule

Never rely only on hiding UI.

For example:

```
</> TypeScript

{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```



This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http

POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



We can do:

```
</> TypeScript

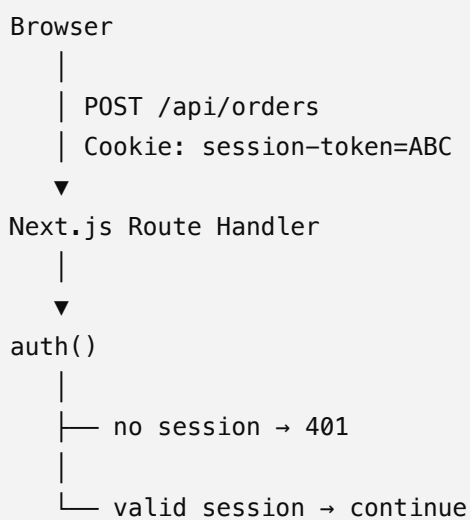
import { auth } from "@auth";
import { NextResponse } from "next/server";

export async function POST(request: Request) {
  const session = await auth();
```



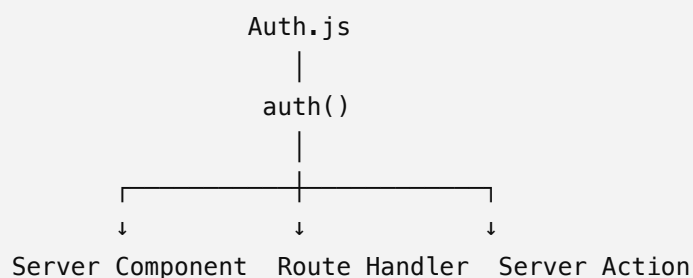
```
if (!session) {  
  return NextResponse.json(  
    { error: "Unauthorized" },  
    { status: 401 }  
  );  
}  
  
// User is authenticated  
// Continue processing order...  
  
return NextResponse.json({  
  success: true,  
});  
}
```

Now:



8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

</> TypeScript



```
const session = await auth();
```

Route Handler

</> TypeScript



```
const session = await auth();
```

Server Action

</> TypeScript



```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

</> TypeScript



```
"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

```
1. LOGIN
  signIn()
    ↓
  OAuth
    ↓
  callback
    ↓
  session created

2. READING SESSION
  auth()
    ↓
  "Who is currently logged in?"

3. PROTECTING ACTIONS
  auth()
    ↓
```



```
authenticate
  ↓
authorize
  ↓
perform operation
```

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

</> TypeScript

```
callbacks: {
  async jwt() {},
  async session() {},
  async signIn() {},
  async redirect() {}
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥

📄 👍 ⬆️ ↺ ...

how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

🔒 Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	🛡️ Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions

#	Topic	What you'll learn
7.7	 Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	 Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	 Authorization / RBAC	Roles, permissions, admin/user access
7.10	 Session Strategies	Database sessions vs JWT sessions
7.11	 CSRF + SameSite	Why state-changing requests need protection
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 🍌 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization** section complete.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

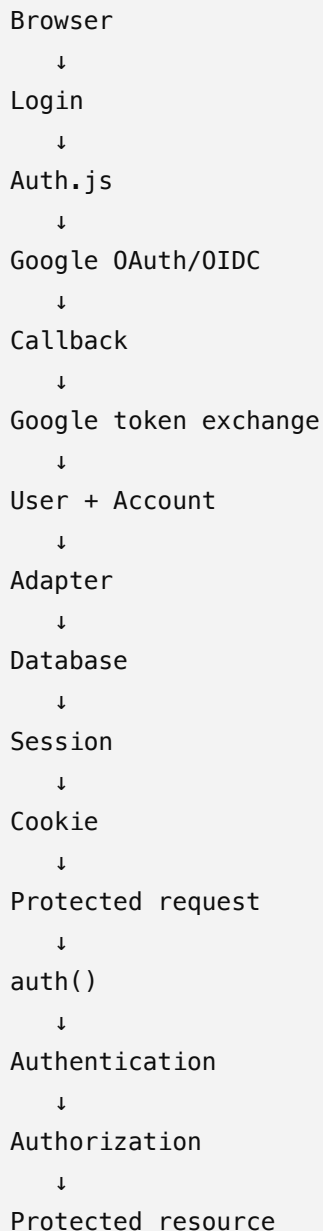
AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD; Browser --> Login; Login --> Authjs[Auth.js]; Authjs --> Google[Google OAuth/OIDC]; Google --> Callback; Callback --> GoogleToken[Google token exchange]; GoogleToken --> User[User + Account]; User --> Adapter; Adapter --> Database; Database --> Session; Session --> Cookie; Cookie --> ProtectedRequest[Protected request]; ProtectedRequest --> auth[auth()]; auth --> Authentication; Authentication --> Authorization; Authorization --> ProtectedResource[Protected resource];
```

Browser

↓

Login

↓

Auth.js

↓

Google OAuth/OIDC

↓

Callback

↓

Google token exchange

↓

User + Account

↓

Adapter

↓

Database

↓

Session

↓

Cookie

↓

Protected request

↓

auth()

↓

Authentication

↓

Authorization

↓

Protected resource

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.